

Comparative Analysis of Dijkstra's, Spira's and a new forward-backward variant of SSSP Algorithms

Abstract

This report presents the implementation and evaluation of three algorithms for the single-source shortest paths problem: Dijkstra's Algorithm, Spira's Algorithm, and a new forward-backward variant of Dijkstra's and Spira's single-shortest paths algorithm that exploits the notion of pertinent edges. The experiments measure how graph topology and varying edge-weight distributions influence algorithmic performance. The results confirm the theoretical claims of the forward-backward variant algorithm on endpoint-independent model with exponential and uniform random edge weight distributions, where a large fraction of non-competitive edges can be pruned effectively. In contrast, the algorithm shows limited benefits on sparse real-world graph networks and complete graphs with geometric weight distribution, where most edges are pertinent and pruning is ineffective. These findings highlight the strengths and limitations of pertinence-based shortest-path algorithms motivating the use of complementary pre-processing techniques for practical applications.

1 Introduction

The single source shortest path problem (SSSP) is a graph optimization problem in computer science where vertices represent entities and edges represent costs, distances, or weights. It can be illustrated through a transportation example. Given a set of cities (vertices) and distances between them (weighted-edges), the goal is to compute the shortest possible routes from a chosen source city to all other cities. It is a polynomial time solvable problem, making it computationally expensive to find optimal solutions for large graph instances. A closely related variant of the

problem is the single-pair shortest path ($s - t$) problem, where the objective is to compute the shortest path between a single source city and a specific destination city. This variant is relevant in practical applications such as navigation systems where a single route is queried rather than distances to all vertices.

We study three algorithms that address both SSSP and $s - t$ problems. We begin with the classical Dijkstra's algorithm [1] which runs in $\mathcal{O}(m + n \log n)$ time. We then consider the Spira's algorithm [3] which improves on Dijkstra's by scanning edges lazily and achieves a runtime of $\mathcal{O}(n \log^2 n)$ when implemented with a comparison-based priority queue. Finally, we examine a new forward-backward variant of Dijkstra's and Spira's algorithm [5] which achieves a runtime of $\mathcal{O}(n)$ with high probability under certain graph models. The algorithms are implemented and evaluated on dense (complete) and sparse graph instances under varying edge weight distributions.

The report is structured as follows: Section 2 introduces the single source shortest path problem and other preliminaries. Section 3 presents the three algorithm approaches and their implementations. Sections 4 and 5 present experimental results and draw conclusions. Section 6 is the appendix that contains additional data tables and plots.

2 Preliminaries

In this section, we begin with a formal definition of a graph and the priority queue data structure, followed by a formal description of the Single-Source Shortest Path (SSSP) problem and the Single-pair ($s - t$) problem. We then introduce the notion of pertinent edges and conclude with the probabilistic endpoint-independent model used

in our analysis.

2.1 Graph

A graph G is formally defined as an ordered pair $G := (V, E)$, where V represents a set of vertices and $E \subseteq V \times V$ represents the set of all undirected edges. In this report, we consider two graph classes. First, a complete (dense) graphs with undirected $\binom{V}{2}$ edges and connected sparse undirected graphs with a number of edges only slightly larger than the number of vertices. The graphs have a non-negative weight function $c : E \rightarrow \mathbb{R}^+$ defined on it's edges. Throughout this report, the terms edge weight, edge cost, and distance are used interchangeably to denote this non-negative value associated with an edge.

2.2 Single Source Shortest Path

Given a weighted graph $G = (V, E)$ with a non-negative cost function $c : E \rightarrow \mathbb{R}^+$ and a designated source vertex $s \in V$, the goal of the SSSP problem is to compute the shortest-path distance $d(s, v)$ from s to every other vertex $v \in V \setminus \{s\}$. The output of an SSSP algorithm is a *shortest-path tree* rooted at s .

A *shortest-path tree* (SPT) of G is a spanning tree rooted at s such that for every vertex $v \in V$, the unique path from s to v in the tree is a shortest path in G .

Equivalently, a tree T with distance labels $d[\cdot]$ is a valid SPT only if the following *SPT condition* holds for every edge $(u, v) \in E$:

$$d[v] \leq d[u] + c[u, v].$$

Any violation of this inequality implies that (u, v) offers a strictly shorter route to v than the one defined by the tree, contradicting the shortest-path property.

2.3 Single Pair Shortest Path

In the Single-pair shortest path problem, also known as source-destination shortest path problem, the goal is to compute the shortest path between a given source vertex s and a target vertex

t . Unlike SSSP, $s - t$ queries allow the algorithm to terminate as soon as the target is settled, thus reducing the explored portion of the graph.

2.4 Pertinent Edges

The new forward-backward variant 3 avoids examining all edges by identifying *pertinent edges* that could possibly violate the SPT condition. *Pertinence* is defined with respect to a threshold value M . In 3, M is set to the *median* of the distances from the source vertex s .

An edge $(u, v) \in E$ is *out-pertinent* with respect to a given source vertex s and threshold M if and only if

$$c[u, v] \leq 2(M - d[u]).$$

An edge (u, v) is *in-pertinent* if and only if

$$c(u, v) < 2(d(v) - M).$$

The out-pertinence condition identifies edges whose weight is small relative to the distance of u from the threshold M . An out-pertinent edge could thus offer a shorter route from u to some vertex beyond M if chosen over other alternative edges.

The in-pertinence condition helps identify edges whose weight is small compared to the gap between $d[v]$ and M . Such an edge could make v closer to s .

Any edge that does not satisfy these conditions automatically is too long to improve the distance from s and is hence considered *non-pertinent*. All edges part of the SPT or $s - t$ result are pertinent.

2.5 Endpoint independent model

The $K_n(\text{Exp}(1))$ model introduced in [5] is a complete graph model of n vertices, and each of the $\binom{n}{2}$ edges receives an independent weight drawn randomly from an exponential distribution with rate 1 or from a uniform distribution on $(0, 1)$.

3 Algorithms and Implementations

This section presents the three single source Shortest Path algorithms. We first describe the algorithms and then go into detail about their implementation to solve the shortest path problem.

3.1 Dijkstra Algorithm

The Dijkstra algorithm [1] is used to find the shortest path from a single source vertex s to all vertices or to a given destination vertex v in a directed or undirected graph.

In each iteration, the algorithm picks an unvisited vertex u from the priority queue P with the lowest cost and relaxes all its outgoing edges $(u, v) \in E$. It computes the tentative distance to each neighbor v of u and updates $dist[v]$ to always be $\min(dist[v], dist[u] + c[u, v])$. If a shorter distance to a neighbor v is found, $dist[v]$ is updated, and v is either inserted into P or its key in P is decreased to the new value. The loop continues until $P \neq \emptyset$.

The algorithm examines each edge at most once. It inserts at most n vertices into the P , performs at most n extract-min operations and at most m decrease-key operations. The running time of Dijkstra's algorithm is $O(m + n \log n)$ with a suitable data structure like the Binary Heap.

Algorithm 1 provides the pseudocode for the Dijkstra algorithm. A lazy update strategy is used to insert a vertex into the priority queue P . When extracting elements, the outdated entries are simply ignored. With this implementation, the overall running time is $O(m \log n)$.

3.2 Spira Algorithm

Spira's algorithm [3] is an improvement on Dijkstra's algorithm. It finds the shortest paths from a vertex s to $\forall v \in V$ potentially without scanning all edges of the graph when the adjacency list of each vertex is given in a *non-decreasing* order of weight. Unlike Dijkstra, Spira relaxes the

Algorithm 1 Dijkstra's Algorithm

Require: Graph $G = (V, E)$, source vertex s

Ensure: Shortest path from s to all vertices

```

1:  $dist[v] \leftarrow \infty$  for all  $v \in V$ 
2:  $prev[v] \leftarrow \text{nil}$  for all  $v \in V$ 
3:  $P \leftarrow$  empty min-priority queue
4:  $dist[s] \leftarrow 0$ 
5: insert  $(0, s)$  into  $P$ 
6: while  $P \neq \emptyset$  do
7:    $(d, u) \leftarrow \text{extract-min}(P)$ 
8:   if  $d \neq dist[u]$  then
9:     continue {stale entry}
10:  end if
11:  for each outgoing edge  $(u, v)$  with weight  $w$  do
12:    if  $dist[u] + w < dist[v]$  then
13:       $dist[v] \leftarrow dist[u] + w$ 
14:       $prev[v] \leftarrow u$ 
15:      insert  $(dist[v], v)$  into  $P$ 
16:    end if
17:  end for
18: end while

```

outgoing edges of a vertex one at a time by maintaining a pointer into its out-adjacency list, so that only the next out-going edge is inserted into the priority queue. The priority queue P in Spira holds edges rather than vertices, where the key of an edge (u, v) in P is $d[u] + c[u, v]$. Once an edge is found in P , we know that $d[u]$ is the correct distance from s to u and u is considered settled and added to a set S .

The algorithm iterates as long as $P \neq \emptyset$ and there are vertices still left to be settled. In each iteration, the algorithm extracts an edge (u, v) with the lowest key from P and adds the next outgoing edge (u, v') of u to P with key $d[u] + c[u, v']$ if it exists. Now, If $v \notin S$, then settle v with $d[u] + c[u, v]$ and add the next outgoing edge (v, w) of v to P . Spira's algorithm inserts edge (u, v) into P even if $v \in S$, that means while P can have only one outgoing edge from u , it can have multiple edges to v .

The expected number of edges scanned by the algorithm is $O(n \log n)$ in the endpoint-

Algorithm 2 Spira's Algorithm

Require: Graph $G = (V, E)$, source vertex s **Ensure:** Shortest distance from s to all vertices

```

1:  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ 
2:  $\text{prev}[v] \leftarrow \text{nil}$  for all  $v \in V$ 
3:  $\text{dist}[s] \leftarrow 0$ 
4:  $S \leftarrow \{s\}$ 
5:  $P \leftarrow \emptyset$ 
6: FORWARD( $s$ )
7: while  $P \neq \emptyset$  and  $|S| \neq |V|$  do
8:    $(u, v) \leftarrow \text{extract-min-edge}(P)$ 
9:   FORWARD( $u$ )
10:  if  $v \notin S$  then
11:     $\text{dist}[v] \leftarrow d[u] + c[u, v]$ 
12:     $\text{prev}[v] \leftarrow u$ 
13:     $S \leftarrow S \cup \{v\}$ 
14:    FORWARD( $v$ )
15:  end if
16: end while
17:
18: function FORWARD( $u$ ):
19:   $v \leftarrow \text{next}(\text{Out\_Adj}[u])$ 
20:  if  $v \neq \text{nil}$  then
21:     $\text{insert}(P, (u, v), d[u] + c[u, v])$ 
22:  end if
```

independent model. With a binary-heap based P , the total running time of the algorithm would be $O(n \log^2 n)$. Algorithm 2 provides the pseudo code for the Spira approach.

3.3 Forward Backward Variant

The new forward-backward variant algorithm [5] improves efficiency by identifying and relaxing only pertinent edges 2.4 in two stages. The algorithm uses a threshold M to determine pertinent edges. Like Spira[3], the algorithm uses a sorted out-adjacency list and a sorted in-adjacency list. It maintains a priority queue P as Spira 3.2 for edge relaxations and a priority queue Q to maintain potential pertinent edges in the second stage, and the key for each item is the cost $c[u, v]$ of an edge $(u, v) \in E$. Each vertex v of an edge $(u, v) \in Q$ has at most one incoming edge in Q at a

time. In this algorithm, *forward scan* identifies and inserts a pertinent edge outgoing from u into P and *backward scan* inserts the next incoming edge (u, v) of a vertex v into Q . Vertices are marked as *active* $[u] = \text{false}$ unless a pertinent edge is found.

Algorithm 3 New Forward-Backward Algorithm

Require: Graph $G = (V, E)$, source vertex s **Ensure:** Shortest distance from s to all vertices

```

1:  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ 
2:  $\text{prev}[v] \leftarrow \text{nil}$  for all  $v \in V$ 
3:  $\text{out}[v] \leftarrow \text{true}$  for all  $v \in V$ 
4:  $\text{Req}[v] \leftarrow \emptyset$  for all  $v \in V$ 
5:  $P \leftarrow \emptyset, Q \leftarrow \emptyset, M \leftarrow \infty$ 
6:  $S \leftarrow \{s\}, \text{dist}[s] \leftarrow 0$ 
7: FORWARD( $s$ )
8: while  $P \neq \emptyset$  and  $|S| \neq |V|$  do
9:    $(u, v) \leftarrow \text{extract-min-edge}(P)$ 
10:  FORWARD( $u$ )
11:  if  $v \notin S$  then
12:     $\text{dist}[v] \leftarrow d[u] + c[u, v]$ 
13:     $\text{prev}[v] \leftarrow u$ 
14:     $S \leftarrow S \cup \{v\}$ 
15:    FORWARD( $v$ )
16:    if  $|S| = |V|/2$  then
17:       $M \leftarrow d[v]$ 
18:      for  $w \in V \setminus S$  do
19:        BACKWARD( $w$ )
20:      end for
21:    end if
22:  end if
23:  while  $Q \neq \emptyset$  &  $\text{min}(Q) < 2(\text{min}(P) - M)$  do
24:     $\text{extract-min-edge}(Q)$ 
25:    if  $v \notin S$  then
26:      BACKWARD( $v$ )
27:      REQUEST( $u, v$ )
28:    end if
29:  end while
30: end while
31: Helper Functions added in 4
```

Additionally, the algorithm maintains an adjacency list $\text{Req}[u]$ for requested in-pertinent edges for each vertex u that needs to be scanned for-

ward in the second stage. During any stage, if a vertex has exhausted its outgoing edges or no more out-pertinent edges are found, $out[u]$ is set to false.

In the first stage, the algorithm behaves like Spira where each vertex maintains a pointer to its sorted outgoing adjacency list, and edges are relaxed one at a time in a sequential manner. All edges extracted from P are out-pertinent as M is set to infinity and the out-pertinence condition $c[u, v] \leq 2(M - d[u])$ is satisfied automatically. The first stage lasts until the shortest paths to the first $\lceil n/2 \rceil$ vertices are found and the value of M is determined. Since M is no longer set to infinity, all edges residing in P at the end of the first stage may not be out-pertinent. The edges inserted in the second stage would be pertinent again.

In the second stage, the algorithm behaves like Spira but only on the implicit subgraph of pertinent edges. When the Median distance is set, the lowest incoming edge $(u, v) \forall v \notin S$ is inserted into Q through a *backward* scan. The algorithm then discovers in-pertinent edges (u, v) with the pertinence condition and adds them to $Req[u]$ so they can be requested in the forward scan. In the forward scan, an in-pertinent edge is only requested when no more out-pertinent edges exist for a vertex. If no pertinent edge (u, v) is found, the vertex is considered to be inactive. If a pertinent edge is found, it is inserted into P so it can be relaxed at an appropriate time later and the vertex is considered *active*. Any discovered in-pertinent edge (u, v) should be scanned forward immediately only if the vertex u is settled and has been marked *inactive* previously.

The runtime of this proposed algorithm when run on a $K_n(\text{Exp}(1))$ is given to be $O(n)$ with very high probability in [5] if used with an efficient data structure like the bucket priority queue. With $O(1)$ amortized operations, the algorithm encounters the $O(n)$ lowest costing edges early and never scans further in their probabilistic model, as the adjacency lists are sorted, allowing the implementation of the algorithm in $O(n)$ time with high probability.

Algorithm 4 New Forward-Backward Algorithm

```

1: function FORWARD( $u$ ):
2:   if  $out[u]$  then
3:      $v \leftarrow \text{next}(Out\_Adj[u])$ 
4:     if  $v = \text{nil}$  or  $c[u, v] > 2(M - d[u])$  then
5:        $out[u] \leftarrow \text{false}$ 
6:     end if
7:   end if
8:   if  $!out[u]$  then
9:      $v \leftarrow \text{next}(Req[u])$ 
10:  end if
11:   $active[u] \leftarrow v \neq \text{nil}$ 
12:  if  $active[u]$  then
13:     $\text{insert}(P, (u, v), d[u] + c[u, v])$ 
14:  end if
15:
16: function BACKWARD( $v$ ):
17:   $u \leftarrow \text{next}(In\_Adj[v])$ 
18:  if  $u \neq \text{nil}$  then
19:     $\text{insert}(Q, (u, v), c[u, v])$ 
20:  end if
21:
22: function REQUEST( $u, v$ ):
23:   $\text{append}(Req[u], v)$ 
24:  if  $u \in S$  &  $!active[u]$  then
25:    FORWARD( $u$ )
26:  end if

```

With Binary Heap based Priority Queues, each insert and extract-min operation requires $O(\log n)$ time. Since $O(n)$ edges are examined by the algorithm, the resulting runtime is $O(n \log n)$ 3.

3.4 Implementation Details

The three algorithms, Dijkstra, Spira, and the new Forward-Backward variant have been implemented in C++. They are exposed via a C++ CLI for tests, benchmarks, and interactive runs, and via a React frontend. Sparse (OSM) graphs are rendered on a Leaflet map. Dense Graphs are drawn on a Euclidean canvas. The Node server brokers requests to the C++ services, and the React UI handles user interaction and visualization. A source node and optionally a destination

node can be chosen to see and, SPT or an $s - t$ path for any of the algorithms with any edge distribution. The user can choose to see only pertinent edges of a query. A runtime comparison is given as part of the visualization to inform users if Spira or the new forward-backward variant is faster than Dijkstra.

The experimental framework is designed to evaluate the three algorithms based on runtime, the number of edge relaxations, and pertinent edges found for SPT and $s - t$ queries on different graph instances. Each output is verified by comparing the costs of the paths from a source vertex produced by different algorithms, flagging any output where these distances do not match. For the new forward-backward variant, the edges in the output are checked to be either out-pertinent or in-pertinent, and not both. It also ensures there are no non-pertinent edges are present in the output paths.

4 Experimental Evaluation

4.1 Data and Hardware

Experiments were run on an HP laptop with an Intel Core Ultra 7 155U CPU (12 cores) and 32 GB of main memory. Performance of the algorithms has been analyzed on two datasets:

1. **National TSP Instances:** 15 complete graphs generated with Euclidean Distances as original edge weights [4].
2. **OpenStreetMap Instances:** 11 Road Networks that are sparse graphs with original edge weights as 1 [2].

The largest graph of the TSP Instances has 9,976 vertices with ≈ 50 million undirected edges and is large enough to stress-test the algorithms. Other than original weights, each instance of both datasets is evaluated under:

1. **Uniform random weights** are sampled independently from a uniform distribution on $(0, 1)$.

2. **Exponential weights** sampled from an exponential distribution with parameter $\lambda = 1$.

The use of uniform and exponential random edge weight distribution helps approximate the endpoint-independent model assumed in the theoretical analysis. This allows us to evaluate whether the probabilistic performance guarantees of the algorithms carry over to practical implementations. Results of all graph instances except the six largest TSP instances are averaged over 100 queries. The large instances are averaged over 50 queries of SPT and $s - t$.

4.2 Experimental Results

The evaluation is focused on the runtime performance of the algorithms, the number of edge relaxations in different graph instances, and the ratio of pertinent edges found by the new forward-backward variant under different graph settings. For all three algorithms [1], [3], [5], the plots and data tables represent the average over all single-source shortest path queries and 100 $s - t$ queries on each instance. All plots and data are included in the appendix.

4.2.1 Runtime Analysis

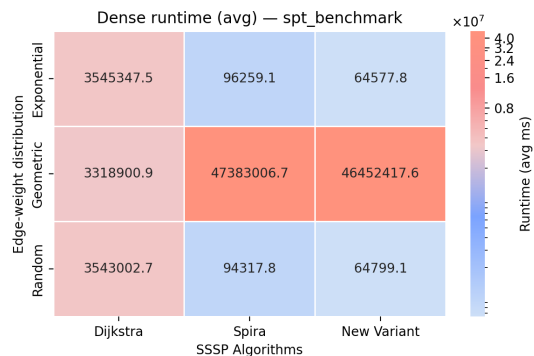


Figure 1: Average runtime heatmap for Dijkstra, Spira and the new forward-backward variant on Dense Graphs with Geometric, Exponential and Uniform random edge-weight distributions (SPT Benchmark).

In dense graphs, with both uniform random and exponential edge weight distributions, Spira and the new forward-backward variant outperform Dijkstra in both SPT and source-destination benchmarks. This improvement stems from the small number of pertinent edges, which allows the algorithms to avoid relaxing most edges in the dense graph. In contrast, with geometric weights, Spira and the new forward-backward variant become up to 10-20x slower than Dijkstra in SPT benchmark and 5-12x slower in the source-destination benchmark. Additionally, Spira is observed to be the fastest in $s - t$ queries for the above endpoint models 4.

In sparse graphs, Dijkstra outperforms Spira and the new forward-backward variant under all edge-weight distributions in SPT 12 and srcdst benchmarks 14. This performance is stable across all weight distributions, indicating that sparsity is the dominating factor not the weight structure.

4.2.2 Edge Relaxations

On dense graphs, the new forward-backward variant prunes more than 98% of the edges as the average number of relaxations is hardly 0.02 as given in table 1. With Geometric weight distribution, The algorithm relaxes slightly more edges (35%). Spira has 2% to 4% more edge relaxations than the new forward-backward variant in dense graphs with geometric edge weight and uniform random edge weight distribution. It performs moderately with exponential edge weight distribution. As graph size increases, figure 7 the ratios grow, indicating that the benefits of the new forward-backward variant is more pronounced on larger dense graph instances.

From 5 we see that 99% of the edges are relaxed in SPT queries with no difference between the weight distributions. In $s - t$ queries, the relaxation ratio drops to 45% for all edge weight distributions. This reduction stems from early termination and not from pruning. From figure 16 and figure 17 show that Spira and new forward-backward variant relax essentially the

same number of edges on sparse graphs.

4.2.3 Pertinence Ratio

In dense graphs, geometric structure amplifies pertinence asymmetry, while random and exponential weights produce balanced and rapidly diminishing pertinence sets as seen in table 2. For graphs with geometric edge weights in figure 9, out-pertinent edges consistently account for 20–25% of all edges, whereas in-pertinent edges remain around 10–15% and occasionally drop much lower. In contrast, for exponential and random weight distributions, both out-pertinent and in-pertinent counts rapidly converge toward zero as the graph size grows, and the two curves shrink at roughly the same rate.

In Sparse Graphs, Table 6, both out-pertinent and in-pertinent edges are nearly equal in number and almost all edges become pertinent in SPT queries regardless of weight distribution. It is observed that in $s - t$ queries, the magnitude of pertinent edges is reduced by a factor of 2 and out-pertinent edges dominate slightly. Nevertheless, sparse graphs are the worst-case scenario for pertinence filtering.

Tables 3 and 7 summarizes that the edges relaxed by the new forward-backward SSSP algorithm are essentially only pertinent edges.

5 Discussion & Conclusion

The comparative evaluations of Dijkstra, Spira and the New forward-backward variant of SSSP algorithms demonstrate that the pertinence based shortest-path algorithm is highly dependent on graph density and structural properties, rather than edge-weight distribution alone. Overall, the results confirm the theoretical claims of the new forward-backward variant 3 algorithm on endpoint models with exponential and uniform random edge weight distributions while highlighting its limitations on real-world sparse graph networks.

In the endpoint model with Random and Exponential edge-weight distributions, most edges

are clearly non-competitive and are eliminated by the pertinence tests, leading to substantial runtime improvements. However, for dense graphs with geometric edge weights, a large fraction of edges may be classified as pertinent because edge weights depend on vertex positions, resulting in many similarly small-weight edges between nearby vertices. As a result, pruning becomes ineffective, and the additional overhead of pertinence checks causes Dijkstra to be significantly faster than the other two algorithms.

Sparse graphs exhibit long average shortest-path trees. Due to their low degree and elongated geometry, sparse graphs provide few alternative routes. Hence, not many edges are pruned, and almost every edge is relaxed as Dijkstra does. The runtime is dominated by the additional overhead of pertinence checks.

These observations suggest that the sparse road networks lack the density required for effective pruning, whereas the strong performance of the new forward-backward variant on endpoint models indicates that pertinence-based filtering is effective when shortest-path trees are shallow and only a small fraction of edges lie near or span the median distance. This insight provides motivation for combining such algorithms with pre-processing techniques such as Contraction Hierarchies to densify the graph structure.

The threshold M is chosen to be the median distance for runtime in the new forward-backward SSSP Variant 3. It would be interesting to explore whether alternative or adaptive threshold choices for M could help prune the number of pertinent edges, particularly in sparse graphs and dense graphs with geometric edge weights.

References

- [1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.
- [2] OpenStreetMap contributors. Planet dump retrieved from <https://planet.osm.org>. <https://www.openstreetmap.org>, 2017.
- [3] P. M. Spira. A new algorithm for finding all shortest paths in a graph of positive arcs in average time $o(n^2 \log^2 n)$. *SIAM Journal on Computing*, 2(1):28–32, 1973. doi: 10.1137/0202003.
- [4] University of Waterloo. World tsp instances. <https://www.math.uwaterloo.ca/tsp/world/countries.html>, 2025. Accessed: March 2025.
- [5] D. B. Wilson and U. Zwick. A forward-backward single-source shortest paths algorithm. In *Proceedings of the 2013 IEEE 54th Annual Symposium on Foundations of Computer Science (FOCS)*, FOCS ’13, pages 707–716, USA, 2013. IEEE Computer Society. ISBN 9780769551357. doi: 10.1109/FOCS.2013.81. URL <https://doi.org/10.1109/FOCS.2013.81>.

A.1 Visualization

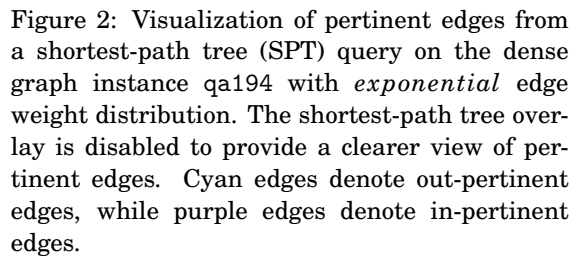
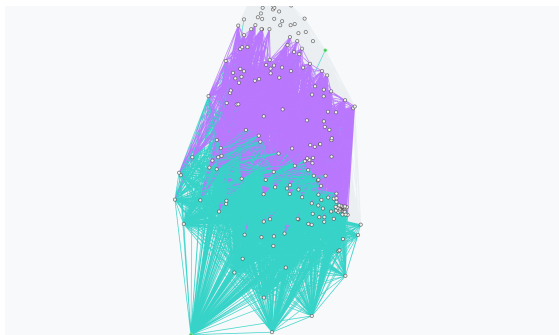


Figure 3: Visualization of pertinent edges from an $s - t$ query on the dense graph instance qa194 with *Geometric* edge weight distribution. The shortest-path overlay is disabled to provide a clearer view of pertinent edges. Cyan edges denote out-pertinent edges, while purple edges denote in-pertinent edges.



Edge-weight distribution	Dense runtime (avg) — src_dst_benchmark		
	Dijkstra	Spira	New Variant
Exponential	1694953.8	12680.7	25940.0
Geometric	1640526.9	10350660.1	13254883.7
Random	1784076.6	13061.9	26897.2

Figure 4: Heatmap of average runtimes for Dijkstra, Spira, and the New Variant on dense graphs under different edge-weight distributions (SRC-DST benchmark).

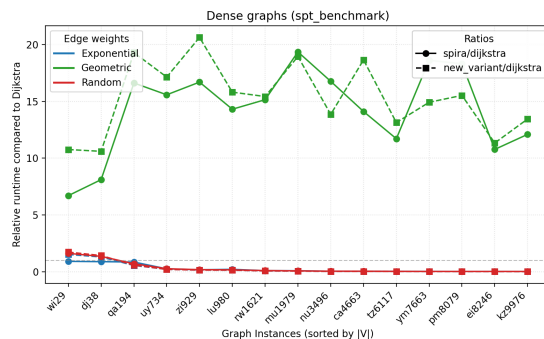


Figure 5: Relative runtime ratios of Spira and the New Variant compared to Dijkstra for SPT queries on dense graphs under exponential, geometric, and uniform random edge-weight distributions.

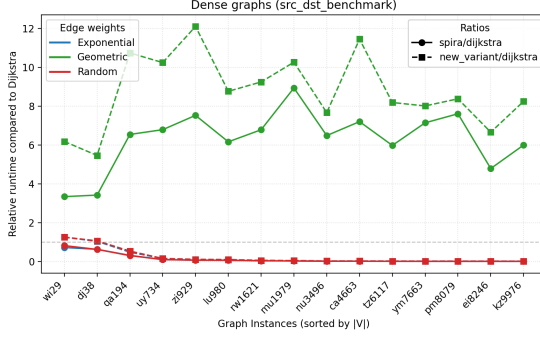


Figure 6: Relative runtime ratios of Spira and the New Variant compared to Dijkstra for $s-t$ queries on dense graphs under exponential, geometric, and uniform random edge-weight distributions.

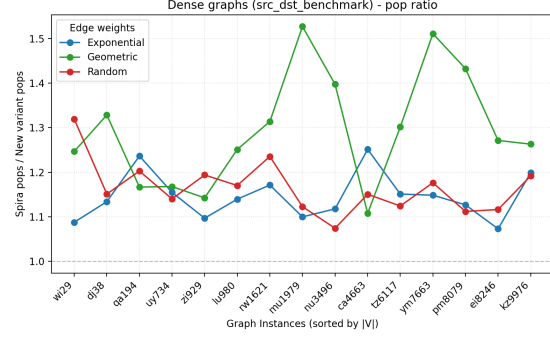


Figure 8: Ratio of priority-queue pops performed by Spira relative to the New Variant for $s-t$ queries on dense graphs under exponential, geometric, and uniform random edge-weight distributions.

A.3 Edge Relaxation plots

Edge weight distribution	spt	src_dst
Exponential	0.01	0.004
Geometric	0.35	0.11
Random	0.01	0.004

Table 1: Average fraction of edges popped by the forward-backward variant, computed over all SPT and $s-t$ queries per instance and then averaged across all instances for each edge-weight distribution.

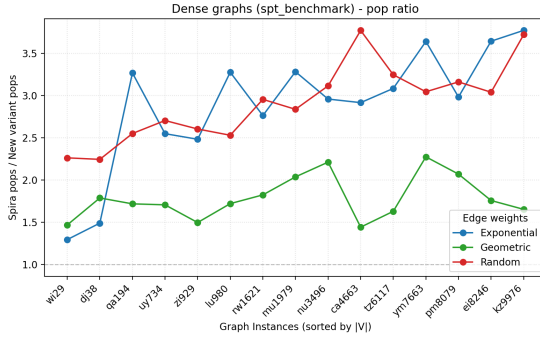


Figure 7: Ratio of priority-queue pops performed by Spira relative to the New Variant for SPT queries on dense graphs under exponential, geometric, and uniform random edge-weight distributions.

A.4 Pertinence Ratios

Edge weight distribution	spt	src_dst
Exponential	2.29	19.64
Geometric	30.39	17.54
Random	2.22	19.28

Table 2: Average out-pertinent to in-pertinent edge count ratio for the forward-backward variant, computed over all SPT and $s-t$ queries per instance and then averaged across all dense graph instances for each edge-weight distribution.

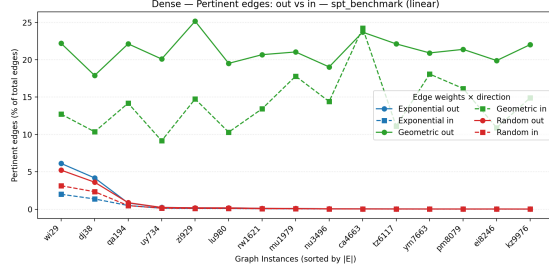


Figure 9: Percentage of pertinent edges (out-pertinent and in-pertinent) for SPT queries on dense graphs under different edge-weight distributions.

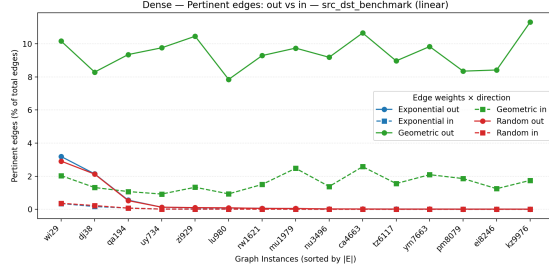


Figure 10: Percentage of pertinent edges (out-pertinent and in-pertinent) for $s-t$ queries on dense graphs under different edge-weight distributions.

Edge weight distribution	spt	src_dst
Exponential	1.06	1.006
Geometric	1.00	0.997
Random	1.07	1.01

Table 3: Average ratio of edge pops to pertinent edges across all files for each edge-weight distribution in dense graphs. Values near 1 mean edge pops closely match pertinent-edge counts.

Edge weight distribution	spt	src_dst
Exponential	2.54	0.99
Geometric	1331.52	412.22
Random	2.56	1.0

Table 4: Average ratio of pertinent edges to the number of vertices (n), computed over all SPT and $s-t$ queries per dense graph instance and averaged across all instances for each edge-weight distribution.

B Results on Sparse Graphs

B.1 Visualization

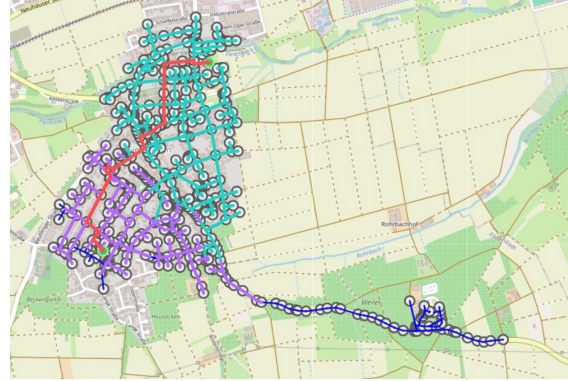


Figure 11: Visualization of pertinent edges from an $s-t$ query on the sparse graph instance OSM1 with exponential edge weight distribution. Cyan edges denote out-pertinent edges, while purple edges denote in-pertinent edges. Blue edges are non-pertinent. Red edges are part of the shortest-path result.

B.2 Runtime Plots

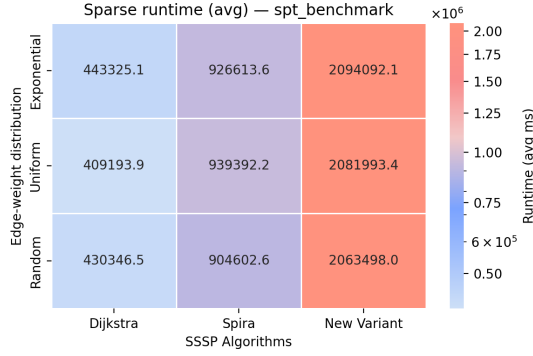


Figure 12: Heatmap of average runtimes for Dijkstra, Spira, and the New Variant on sparse graphs under different edge-weight distributions (SPT benchmark).

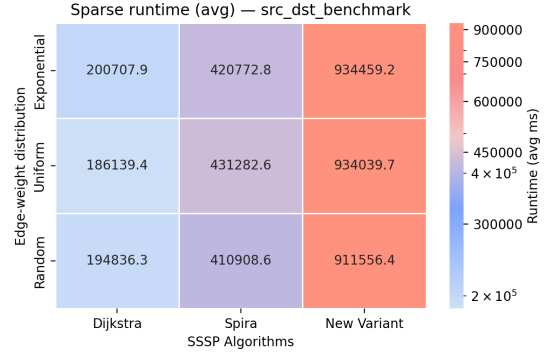


Figure 14: Heatmap of average runtimes for Dijkstra, Spira, and the New Variant on sparse graphs under different edge-weight distributions (SRC-DST benchmark).

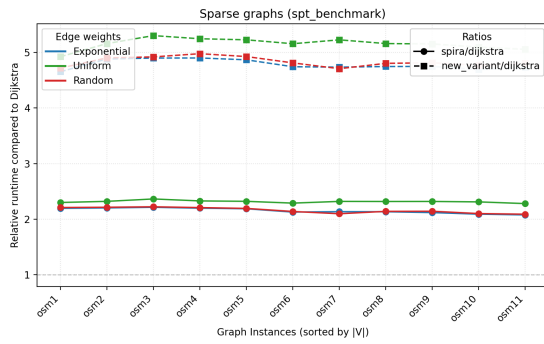


Figure 13: Relative runtime ratios of Spira and the New Variant compared to Dijkstra for SPT queries on sparse graphs under exponential, and uniform random edge-weight distributions and uniform edge weight of 1.

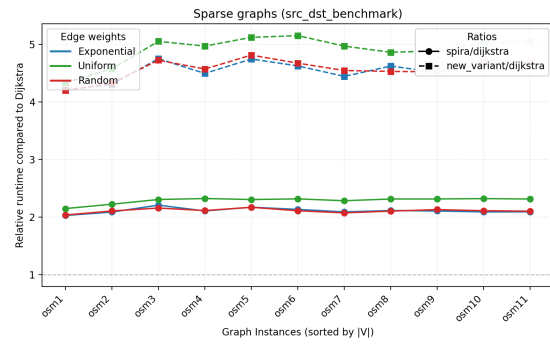


Figure 15: Relative runtime ratios of Spira and the New Variant compared to Dijkstra for $s-t$ queries on sparse graphs under exponential, and uniform random edge-weight distributions and uniform edge weight of 1.

B.3 Relaxation Ratios

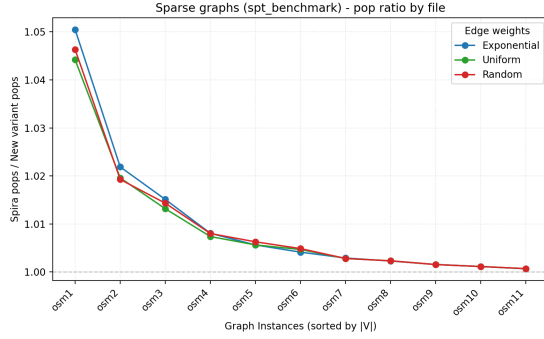


Figure 16: Ratio of priority-queue pops performed by Spira relative to the New Variant for SPT queries on sparse graphs under exponential, uniform random edge-weight distributions and uniform weights of 1.

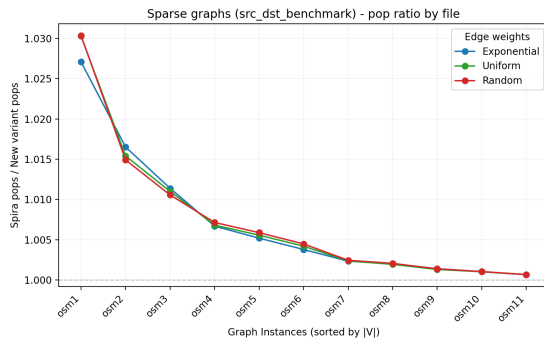


Figure 17: Ratio of priority-queue pops performed by Spira relative to the New Variant for $s-t$ queries on sparse graphs under exponential, uniform random edge-weight distributions and uniform weights of 1.

Edge weight distr.	spt	st
Exponential	0.99	0.46
Uniform	0.99	0.45
Random	0.99	0.45

Table 5: Average fraction of edges popped by the forward-backward variant, computed over all SPT and $s-t$ queries per instance and then averaged across all sparse instances for each edge-weight distribution.

B.4 Pertinence Ratios

Edge weight distribution	spt	src_dst
Exponential	1.023	10.556
Uniform	1.023	10.665
Random	1.023	10.562

Table 6: Average out-pertinent to in-pertinent edge count ratio for the forward-backward variant, computed over all SPT and $s-t$ queries per instance and then averaged across all sparse graph instances for each edge-weight distribution.

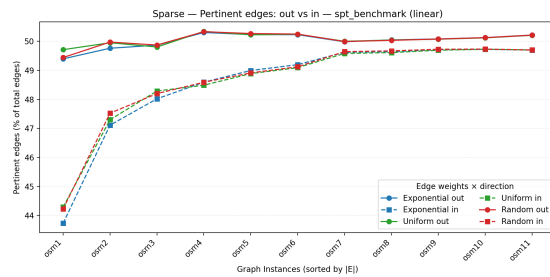


Figure 18: Percentage of pertinent edges (out-pertinent and in-pertinent) for SPT queries on sparse graphs under different edge-weight distributions.

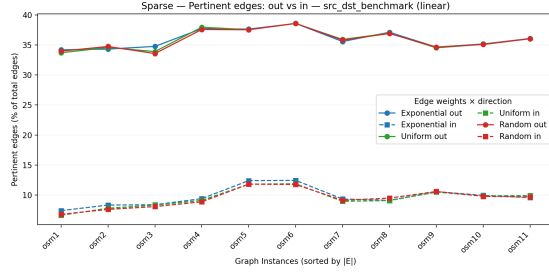


Figure 19: Percentage of pertinent edges (out-pertinent and in-pertinent) for $s - t$ queries on sparse graphs under different edge-weight distributions.

Edge weight distr.	spt	src_dst
Exponential	1.004	1.003
Random	1.003	1.003
Uniform	1.003	1.002

Table 7: Average ratio of edge pops to pertinent edges across all files and benchmarks for each edge-weight distribution in sparse graphs. Values near 1 mean edge pops closely match pertinent-edge counts.

Edge weight distribution	spt	src_dst
Exponential	2.12	0.98
Random	2.12	0.97
Uniform	2.12	0.97

Table 8: Average ratio of pertinent edges to the number of vertices (n), computed over all SPT and $s-t$ queries per sparse graph instance and averaged across all instances for each edge-weight distribution.